FastSpeech 2 for English and Telugu

Technical report for the SLFI Speech Processing Lab assignment.

Author: Angshuman Chakravertty

---

1. Implementation approach

1.1 What I started from

I used ming024/FastSpeech2 as the reference implementation, which is the one linked in the assignment. It is a faithful reimplementation of the paper and it already has the pieces the assignment asks me to analyse, in particular a variance adaptor that predicts duration, pitch and energy at the phoneme level.

The repository was last updated in October 2023 and pins torch 1.7.0, numpy 1.19.0 and librosa 0.7.2. None of those install cleanly on a current machine, so a fair amount of my work was porting it rather than rewriting it. I chose to patch the call sites against current libraries instead of trying to recreate a 2023 environment, because the trained checkpoints have to be usable by whoever evaluates this.

The model has 35,167,809 parameters, which I report from the training log rather than from the paper.

1.2 Architecture components I used unchanged

I kept the model itself as it is in the reference, because the assignment is about adapting and evaluating FastSpeech 2 rather than redesigning it. Used as is:

- The phoneme encoder, four feed-forward Transformer blocks with multi-head self-attention and a 1D convolutional feed-forward network.
- The variance adaptor, with separate predictors for duration, pitch and energy. Each is two 1D convolution layers with ReLU, layer normalisation and dropout, followed by a linear projection. Duration is predicted in the log domain and rounded. Pitch and energy are quantised into 256 bins and embedded, then added to the hidden sequence.
- The length regulator, which expands the phoneme sequence to frame length using the predicted durations.
- The mel decoder, another four feed-forward Transformer blocks, plus the 5-layer postnet.
- The loss, which is L1 on the mel spectrogram before and after the postnet, plus MSE on duration in the log domain, pitch and energy.
- HiFi-GAN as the vocoder, using the universal checkpoint that ships with the reference repository.

1.3 Modifications common to both languages

These are changes I had to make for the code to run at all, or to produce the deliverables the assignment asks for.

Device handling. The reference hardcodes `torch.device("cuda" if torch.cuda.is_available() else "cpu")` in five separate files. I replaced all of them with a single `device_utils.py` that picks CUDA, then Apple MPS, then CPU, and can be overridden with the `FS2_DEVICE` environment variable. This let me develop and validate on a laptop and train on a GPU without editing anything.

librosa API. `pad_center` became keyword-only in librosa 0.10, so `pad_center(x, size)` raises `TypeError`. Fixed in `audio/stft.py` and `audio/audio_processing.py`.

Checkpoint loading. torch 2.6 changed the default of `weights_only` to `True`, which makes the reference's `torch.load` calls fail. I set `weights_only=False` explicitly where the file is one I produced.

Vocoder paths. The reference resolves `hifigan/config.json` relative to the current working directory, so it only works if you launch from the repository root. I made it resolve relative to the source file instead, and unzipped the bundled generator checkpoints, which ship as `.pth.tar.zip`.

Config files. The reference splits configuration across three YAML files per dataset. The assignment asks for one `config.yaml` per language, so I merged them and wrote `config_utils.py` to split the merged file back into the three dictionaries the rest of the code expects. It resolves every path key relative to the config file, and takes `--set section.key=value` overrides on the command line. That override mechanism is what lets the same config file work locally and on a hosted GPU where the paths are different.

Loss logging. The reference writes losses only to TensorBoard. The assignment asks for loss plots, so the trainer now appends every logged step to `losses_train.csv` and `losses_val.csv`, and `scripts/plot_losses.py` draws the figures from those. I also made `evaluate()` return its loss means instead of only a printable string.

Wall clock guard. Training runs on a hosted GPU session that is killed at a fixed limit, and the output is discarded when that happens, so an overrunning job loses every checkpoint rather than just the last stretch. `FS2_MAX_HOURS` stops the loop at a clean step, saves, and exits before the cap.

Checkpoints. Each save now also writes a weights-only copy next to the full one. The full checkpoint carries the optimiser state and is about 420 MB, the weights-only copy is about 140 MB, which is what the repository ships.

1.4 Three bugs in the reference implementation

These are not porting issues. They are defects that affect correctness, and I mention them because two of them would have quietly damaged the results.

The variance scalers were updated outside the branch that produced the values. In the preprocessor, the pitch and energy `StandardScaler` instances were being fitted in a code path that ran whether or not the utterance actually had an alignment. Fitting a scaler on values from skipped utterances shifts the normalisation statistics, which then shift every pitch and energy target the model trains on. I moved the updates inside the branch.

The train/validation split was destroyed by a shuffle. The reference builds the split by shuffling the full list of utterances and slicing off the first `val_size` entries. That is fine for a single run, but it means the split is decided at preprocessing time by an unseeded shuffle and cannot be reproduced or

audited. The assignment specifically requires the splits to be separate with no overlap, so I made the preprocessor read the split assignment from the `metadata.json` that my data preparation writes, and fall back to the old behaviour only when that file is absent.

`nn.DataParallel` was applied unconditionally. The reference wraps the model in `DataParallel` regardless of how many devices exist. On a single device this is pointless overhead, and on MPS it fails outright. I now wrap only when there is genuinely more than one CUDA device.

1.5 Modifications for Telugu

This was the bulk of the work, because nothing in the Telugu path existed.

No grapheme-to-phoneme converter exists. The reference supports English via `g2p_en` and Mandarin via pypinyin. For Telugu I wrote a rule-based G2P, `text/telugu_g2p.py`, with a 51-phone inventory. Telugu is an abugida, so the rules it has to handle are:

- Every consonant carries an inherent /a/ unless something cancels it.
- A vowel sign (matra) replaces the inherent vowel.
- The virama (U+0C4D) cancels it, which is how consonant clusters and geminates are written.
- The anusvara (U+0C02) is a nasal whose place of articulation comes from the following consonant, so it has to be resolved to one of ng, ny, nn, n or m rather than mapped to a fixed symbol.

Two checks of the output: పట్టు gives `p a tt tt u`, and నిశ్శబ్దంగా gives `n i sh sh a b d a ng g aa`, where the anusvara has correctly become `ng` before `g`.

No MFA acoustic model exists. Montreal Forced Aligner ships pretrained models for English and Tamil but not Telugu, so English alignment is a download and Telugu is not. I trained a Telugu acoustic model from scratch on the selected corpus, through the standard MFA progression of monophone, then triphone, then LDA, then speaker-adapted training.

No pronunciation dictionary exists. MFA needs every transcript token to be in a dictionary. I generated one from the corpus transcripts with `scripts/build_telugu_lexicon.py`, running each token through the G2P. This took two passes. Stripping only leading and trailing punctuation left 5315 tokens unmatched, because abbreviations such as ఐ.టి.ఐ carry punctuation inside the word. Splitting on internal punctuation as well, and dropping tokens that contain no consonant or independent vowel, got it to 14621 entries with zero out-of-vocabulary tokens. The second filter matters because a stray anusvara had otherwise become its own dictionary entry.

Text normalisation for the test sentences. Test sentence T4 contains a year, an ordinal and a time, none of which the lexicon covers. `text/telugu_norm.py` expands numbers up to 9999, ordinals and times. Three details I had to fix by hand: a bare hundred is వంద rather than ఒకటి వందల, the ordinal form drops the final ు before వ, and the first ordinal is the irregular మొదటి rather than a regular form. Checked on T4, 2026 becomes `r e nn dd u v ee l a`, 12వ becomes `p a n n e nn dd a v a`, and 10:45 becomes `p a d i n a l a bh ai ai d u`.

Symbol table collision. Adding 51 Telugu phones to the shared symbol table produced 411 symbols of which 18 were duplicates, because several Telugu phone names collide with the lowercase pinyin finals already in the table. Duplicate symbols silently break the text-to-sequence mapping, since two

different inputs map to the same index. I deduplicated with `dict.fromkeys`, which preserves order and leaves 393 symbols with all 51 Telugu phones present.

Speaker enumeration. The preprocessor treats every entry in the input directory as a speaker, which means my `metadata.json` would have been read as a speaker directory. Restricted to directories.

1.6 Modifications for English

English needs no new phone inventory and no acoustic model, because MFA ships both. What it needed instead was work on the text, because Svarah is not the kind of English the reference assumes.

No acoustic model to train. Alignment uses the pretrained `english_us_arpa` model and dictionary, which is a download rather than the 80 minutes of training the Telugu model took.

Multi-speaker. The config sets `multi_speaker: True`, since Svarah has 115 speakers against Telugu's one.

The dictionary had to be extended. Svarah is Indian English about government services and digital payments, and against the 208651-entry `english_us_arpa` dictionary **4.53% of its tokens are out of vocabulary and 33.1% of utterances contain at least one**. The gaps are Indian place names, scheme acronyms such as EPFO, ESIC, PRAN and UPI, and a handful of ordinary words the dictionary simply lacks, including "download".

That would not have failed loudly. An unknown word either drops its utterance or aligns against an unknown phone, and at a third of the corpus it would have quietly distorted the phoneme durations the variance adaptor learns from.

`scripts/extend_english_lexicon.py` collects the out-of-vocabulary list and MFA's English G2P supplies pronunciations. The G2P is good on real words, giving karnataka, dosa and download correctly, but it treats every initialism as a word, so it returns "oh tee p" for otp and "uh pee" for upi.

Spelling cannot separate initialisms from ordinary words here, because the short list mixes epfo, cbse and gps with dosa, aloo and dahi. It does not need to. MFA allows several pronunciations per word and the aligner picks whichever fits the audio, so every token of four characters or fewer also gets the letter-by-letter form, as does any longer token with no vowel. A candidate that is wrong is simply never chosen. The result is 212165 entries and zero out-of-vocabulary tokens.

Text normalisation, corrected against measured alignment quality. Three faults, found by reading the utterances that failed to align rather than by guessing. Ordinals were being split, so "7th July" became "seven th july". Four-digit identifiers such as an MPIN were being read as quantities. And years were being read the long way.

The year one is measurable, because MFA writes an `alignment_analysis.csv`. Utterances carrying a year expanded as "one thousand nine hundred and eighty four" aligned with a median phone duration deviation of 3.61, against 2.29 for everything else, where the whole corpus's 90th percentile is 3.72. Those utterances were aligning about as badly as the worst tenth of the corpus, and there were 103 of them. A year is said in two halves, so 1984 is now "nineteen eighty four" and 1905 is "nineteen oh five", gated to 1100-2099 and suppressed when a unit word follows so that "1500 rupees" stays a quantity.

Re-aligning confirms the fix, measured on the 3899 utterances common to both runs:

group	n	before	after	change
carries a year	99	3.609	2.499	-1.111
everything else	3800	2.291	2.303	+0.013

70 of the 99 improved individually. The affected group moved from worse than the corpus 90th percentile to roughly its median, and the rest of the corpus did not move, which is the shape a targeted fix should have.

Corpus-wide the median barely moves, 2.315 to 2.308, because 99 utterances out of 3899 cannot shift it. The per-group split is the number that means anything.

A trap worth recording. `g2p_en` imports cleanly and only fails when called, because it fetches the NLTK `averaged_perceptron_tagger_eng` resource at run time. Two of the four English test sentences contain words that miss the lexicon, "antidisestablishmentarianism" and the deliberately broken "rhyth", so inference reaches `g2p_en` and dies there. The download step is in `requirements.txt` and the README.

> **Status note.** Both corpora are prepared and aligned. Sections 3 to 6 are written once each model has trained, and say so until then.
>
> For anyone reproducing this: Svarah is distributed only through a gated HuggingFace dataset, and there is no mirror. The AI4Bharat GitHub repository, the AI4Bharat dataset API and the AIKosh government portal all point back to the same gated entry, and anonymous access reads the file list but returns 401 on the data itself. Accepting the terms on an account and authenticating is the only route in.

---

2. Datasets

2.1 Telugu, IndicTTS Telugu

I used the `SPRINGLab/IndicTTS_Telugu` HuggingFace release. Two things about it drove most of my decisions.

First, it is **16 kHz**, not the 48 kHz of the original IIT Madras database. Second, it has no speaker ID field, only `gender`, which for IndicTTS means one female and one male speaker.

Speaker selection. I measured both speakers from the audio headers rather than assuming:

speaker	utterances	total	mean length	usable at 1-12 s
female (gender=0)	4043	23.92 h	21.30 s	2.31 h
male (gender=1)	4533	12.79 h	10.16 s	6.86 h

The assignment needs 6 h. The corpus is long-form narration and the female speaker's median utterance is 18.5 s, with the longest file at 151.6 s. Long utterances hurt forced alignment and cost a lot of memory in training, so once I restrict to a length band that FastSpeech 2 can train on, the female speaker yields only 2.31 h. The male speaker yields 6.86 h. So I used the male speaker and built a single-speaker model.

I verified the gender mapping rather than trusting it. Across all 8576 rows, `gender=1` corresponds exactly to the files named `telugumale`.

Subsampling strategy. In order:

1. Keep only the male speaker.
2. Keep utterances between 1 and 15 seconds. I started at 1-12 s, which gave 6.86 h, but that turned out not to be enough once trimming was accounted for, so I widened the band. At 1-15 s there are 3806 utterances and 8.11 h.
3. Shuffle with seed 1234.
4. Resample to 22.05 kHz, trim leading and trailing silence, and peak normalise to 0.95.
5. Fill the 1 h validation budget first, then the 5 h training budget, using the duration **after** trimming.

Step 5 needs explaining. My first attempt budgeted on each file's raw duration and produced a corpus that MFA measured at 5.13 h rather than 6 h, because silence trimming removes about 14% of this corpus. Budgeting before trimming undershoots by roughly that much, and the delivered split would have been 4.3 h and 0.85 h instead of 5 h and 1 h. Measuring the trimmed audio and only then assigning the file to a budget fixes it exactly.

I resampled up to 22.05 kHz to match the pretrained HiFi-GAN vocoder. This does not lose anything the model sees, because the mel front end uses `mel_fmax: 8000` and a 16 kHz recording already carries everything up to 8 kHz. The real caveat is narrower: the vocoder reconstructs the 8 to 11 kHz band from a mel that never contained it. That applies to both languages equally.

Train and validation split. Disjoint by construction. The shuffle is seeded, the validation budget is filled first, and a file can only land in one budget, so there is no overlap to check for afterwards. The split assignment is written to `metadata.json` and the preprocessor reads it, so the split is fixed in the data rather than decided at preprocessing time.

split	utterances budgeted	duration	after alignment
train	2670	5.000 h	2669
validation	574	1.000 h	573
total	3244	6.000 h	3242

Two utterances failed to align and are absent from the feature set rather than silently mismatched. Both are long run-on sentences of comma-separated vocatives where the aligner could not fit the transcript to the audio. That is a 0.06% loss and the split durations are still 5.00 h and 1.00 h.

Preprocessing. Per utterance: resample, trim, peak normalise, force align with the Telugu acoustic model I trained, read phoneme durations from the TextGrid, extract F0 with pyworld `dio` refined by `stonemask`, extract a mel spectrogram and frame energy, average pitch and energy over each phoneme's frames, then normalise pitch and energy with statistics computed over the training set.

The resulting statistics are pitch mean 187.21 Hz with standard deviation 45.45, and energy mean 43.86 with standard deviation 33.79.

A note on that pitch mean. 187 Hz is high for a speaker labelled male and looks like the classic pyworld octave-doubling failure, where the estimator locks onto the first harmonic. I checked, and it is not. The distribution is unimodal, with a median of 185.7 Hz and quartiles at 154.5 and 219.0, and octave doubling leaves a second mode near twice the first. Four independent estimators also agree:

estimator	settings	median of per-file medians
dio + stonemask	floor 71, ceiling 800 (the default)	201.2 Hz
dio + stonemask	floor 60, ceiling 300	198.0 Hz
harvest	floor 60, ceiling 300	197.4 Hz
librosa yin	fmin 60, fmax 400	206.6 Hz

Narrowing the ceiling moves the estimate by 3 Hz, and harvest, which is much more resistant to octave error than dio, agrees to within 4 Hz. So the speaker really does sit near 190 to 200 Hz. That is in the usual adult female range and above the usual adult male range, so the corpus's own gender label looks questionable, even though the label-to-filename mapping is consistent. Either way the extraction is correct and the pitch targets are sound.

2.2 English, Svarah

Svarah is an Indian-accented English ASR benchmark: 9.6 hours of transcribed audio from 117 speakers across 65 districts in 19 states, released under CC BY 4.0 (Interspeech 2023, arXiv 2305.15760).

Two facts about it shape the English side. It is only 9.6 h in total, against the 6 h the assignment asks for, so there is far less room to subsample than on the Telugu side. And it has 117 speakers, so a single-speaker model is not possible and the English config is multi-speaker.

Svarah is an ASR evaluation benchmark, not a TTS corpus. It contains spontaneous as well as read speech, recording conditions vary across speakers, and it was never built for synthesis. The assignment fixes the dataset, so I use it, but I expect this to show up in the results and I treat it as a limitation rather than a surprise.

Speakers had to be recovered. The schema is `audio_filepath`, `duration`, `text` and eight demographic fields. There is no speaker column, and a multi-speaker model needs one.

The filename looks like it carries it. It does not. Measured over all 6656 utterances:

grouping	distinct values
leading recording id	2938
the `fNNNN` field	3232
recording id and `fNNNN` together	3232
the eight demographic fields	**115**

The paper documents 117 speakers, so both filename fields are per-recording at roughly two utterances each. The `f` prefix is not a gender marker either, it appears on 3579 female and 3077 male rows alike. The demographic tuple is therefore the speaker proxy. 115 against a documented 117 means two pairs of speakers share an identical tuple and get merged, which I record rather than hide. The grouping is otherwise well behaved, a median of 47 utterances and 227 s per speaker, with the largest five holding 15.7% of the audio between them.

Subsampling strategy. Keep utterances between 1 and 15 seconds, which leaves 5180 utterances and 7.97 h from the 9.61 h total. Shuffle with seed 1234. Resample to 22.05 kHz, trim, peak normalise to 0.95. Then fill the budgets on post-trim duration, as on the Telugu side.

The split is stratified by speaker, which is the one real difference from Telugu. Telugu is a single speaker, so filling the validation budget first is enough. English has 115, and a multi-speaker model validated on speakers it never trained on measures the wrong thing, so each speaker's first utterance goes to validation before anything else.

split	utterances	duration	speakers
train	3270	5.00 h	115
validation	649	1.00 h	115

All 115 speakers appear in validation, so it covers the full accent range rather than whichever speakers happened to shuffle to the front.

There is much less headroom here than on the Telugu side. Svarah has 9.61 h in total against the 6 h the assignment asks for, so after the length band and trimming the budget is nearly the whole usable corpus. Telugu could afford to discard an entire speaker.

2.3 How much audio the models actually see

"5 hours" is true of the selection, not of the mel frames.

Silence comes off three times. The length band picks utterances, librosa trims each one and the budgets are filled on that trimmed duration, and then the preprocessor's `get_alignment` drops every leading and trailing silence interval that MFA marked, keeping internal pauses as phones.

	selected	after alignment	lost
Telugu train	5.000 h	4.263 h	15%
Telugu validation	1.000 h	0.853 h	15%
English train	5.000 h	3.949 h	21%
English validation	1.000 h	0.797 h	20%

This is not a fault. Removing head and tail silence is what the reference preprocessor does, and training a duration predictor on arbitrary silence would be worse. The selection is exactly the 5 h and 1 h the assignment asks for.

I chose not to inflate the budgets to land on 5 h after alignment, which would need about 6.3 h of training audio per language. Telugu has the room. English does not, since Svarah offers 7.97 h in the 1-15 s band and train plus validation would consume 7.6 h of it on a corpus that is already essentially the whole usable dataset. Treating the two languages identically matters more here than hitting a round number, because section 6 compares them. Both are selected the same way and lose silence to the same code, and the gap between 15% and 21% is a property of the corpora rather than of the method, since Svarah is ASR material with more head and tail silence than studio narration.

2.4 Training configuration

The same for both languages except where noted.

setting	value
sampling rate	22050 Hz
mel channels	80
FFT / window / hop	1024 / 1024 / 256
mel frequency range	0 to 8000 Hz
pitch and energy feature	phoneme level, normalised
batch size	16
optimiser	Adam, betas 0.9 and 0.98, eps 1e-9
warm-up steps	4000
learning rate anneal	at 30000, 50000, 70000, rate 0.3
gradient clipping	1.0
total steps	80000
validation interval	1000 steps
checkpoint interval	10000 steps
vocoder	HiFi-GAN, universal checkpoint
speakers	Telugu single, English multi

I reduced total steps from the reference default of 900000 to 80000. 900000 is not reachable in the time available for this assignment, and at 80000 steps the Telugu model sees about 480 epochs over 2669 utterances, which is enough to train a model of this size on a corpus this small. I note it as a deliberate departure rather than a tuned choice.

Training ran on a Kaggle Tesla T4. Two T4s are offered, but I pinned training to one device, because at batch size 16 `DataParallel` splits into 8 per device for very little gain while adding a failure mode to a long unattended job.

---

3. Training and validation loss curves

Figures are `project/Results/<language>/training_loss.png` and `validation_loss.png`, drawn from the per-step CSVs the trainer writes.

3.1 Telugu

The run reached **step 63675 of 80000** in 11 h 58 m on a Kaggle T4, at about 1.5 steps per second, and was then stopped by Kaggle at its 12 h session limit. That is 384 epochs over 2669 utterances. Checkpoints are written every 10000 steps, so the last complete one is at 60000 and nothing was lost. Section 3.3 explains why the missing 16000 steps do not matter.

Training loss falls cleanly and never becomes unstable. Total goes from 10.30 at step 100 to 0.74, with no spikes, no divergence and no sign of a learning-rate problem at any of the anneal points.

component	step 100	step 63600
total	10.30	0.74
mel	3.83	0.37
mel postnet	3.25	0.37
pitch	1.33	0.021
energy	1.45	0.019
duration	0.44	0.010

Validation tells a completely different story, and it is the main result of this section. Validation loss does not merely stop improving. It gets worse, and it starts getting worse almost immediately.

	best value	at step	value at 63000
total	3.834	**2000**	4.068
mel postnet	0.9243	**6000**	0.9573
pitch	1.075	**1000**	1.2814
energy	0.673	**1000**	0.7971
duration	0.082	1000	0.0749

So the model's best validation score arrives within the first few thousand steps, and the remaining 60000 steps make it worse on every component except duration.

3.2 What is overfitting, and what is not

The components behave differently and lumping them together would hide that.

Pitch and energy overfit severely. At the end, training pitch loss is 0.021 and validation pitch loss is 1.281, a factor of **61**. Energy is 0.019 against 0.797, a factor of **42**. Their validation curves rise steeply over the first 8000 steps and then sit flat. The shape is diagnostic: an untrained predictor outputs something close to the dataset mean, and predicting the mean is a decent strategy under mean squared error. As the predictor learns the training set's actual pitch contours it stops predicting the mean, and because those contours do not generalise from 2669 utterances, validation error goes up. The predictors are memorising prosody.

Duration does not overfit. Validation duration loss improves slightly, from 0.082 to 0.075, and ends at 0.010 on training. Duration is the most learnable of the three, which makes sense: how long a phoneme lasts is largely a property of the phoneme and its neighbours, and that generalises. Pitch is a property of the sentence, the speaker's intent and the discourse, and that does not.

The mel decoder is roughly neutral. Validation mel postnet improves from 0.987 to 0.924 over the first 6000 steps, then drifts back to 0.957. The total drift is 3.6%, which is small, so the acoustic model itself is neither learning much more nor falling apart after step 6000.

This matters for how the rest of the report reads. The variance adaptor plots in section 5 are the output of predictors that fit their training data almost perfectly and generalise badly, so the pitch and energy contours there should be read as plausible-looking prosody rather than as correct prosody for unseen text.

3.3 Why the missing 16000 steps do not change anything

Two independent lines of evidence.

First, the validation curve above is flat from roughly step 30000 onward. The total moves from 4.07 to 4.07 between step 30000 and step 63000.

Second, and more directly, I synthesised all five test sentences from both the 10000-step and the 60000-step checkpoints and compared them. The outputs are nearly identical:

	T1	T2a	T2b	T3	T4
duration at 10000, s	4.54	2.15	2.31	1.93	7.16
duration at 60000, s	4.59	2.10	2.24	1.94	6.95
mean F0 at 10000, Hz	191.8	192.1	178.5	194.3	169.3
mean F0 at 60000, Hz	200.6	197.2	194.2	201.7	175.1

Six times as much training moves the predicted duration of each sentence by 1 to 3% and the mean F0 by at most 9%. Neither checkpoint produces degenerate audio, and neither clips.

I report the 60000-step checkpoint as the model, since it is the most trained one available and its outputs are sound. I am not claiming the extra steps helped. This model converged within about 10000 steps and spent everything after that memorising the training set.

The practical lesson, if this were to be run again, is that 80000 steps was the wrong target for 5 hours of single-speaker audio. Early stopping on validation loss, or a much smaller model, would have been the better choice. I set 80000 by reducing the reference's 900000 to fit the time available, which was a scheduling decision rather than one informed by this corpus, and the curves show it.

3.4 English

English reached **step 73100 of 80000** in exactly 11.00 h on a T4, at about 1.85 steps per second, and stopped itself because `FS2_MAX_HOURS` was reached. That is 361 epochs over 3262 utterances.

The shape is the same as Telugu. Training falls cleanly to 0.778 total, and validation bottoms out early and then drifts the wrong way.

	best value	at step	value at 73000
total	3.436	**2000**	3.608
mel postnet	0.9216	**7000**	0.9650
pitch	0.551	25000	0.5594
energy	0.772	1000	0.8893
duration	0.2148	1000	0.2297

Final training losses are mel 0.371, pitch 0.0157, energy 0.0141, duration 0.0066, so the train-to-validation gap is **36x on pitch** and **63x on energy**. Same memorisation as Telugu.

The difference that matters is duration. Telugu ends at a validation duration loss of 0.075, English at **0.2297**, three times worse, and unlike Telugu it does not improve at all over the run. This is the multi-speaker effect and it is exactly what I expected to see. Telugu is one speaker with one speaking rate, so how long a phoneme lasts is close to a deterministic function of the phoneme and its context. English is 115 speakers who talk at different speeds, and the model has no speaker embedding at inference to tell it whose rate to use, so the best it can do is predict an average that fits nobody.

Pitch is the opposite way round in absolute terms, 0.559 for English against 1.281 for Telugu, but that comparison is not meaningful. Each corpus is normalised by its own statistics, and a 115-speaker pitch distribution is far wider, so the same normalised error means something different in each.

---

4. Observations on the generated speech

All ten wavs are in `project/Results/<language>/test_generated_audios/`.

A limit on this section. I can report what is measurable from the phone sequences, the predicted durations and the waveforms. I have not assessed naturalness or intelligibility by ear, and I am not going to write down impressions I did not form. Judgements like "sounds natural" or "the speaker

sounds breathy" would need listening, and that is the one part of this analysis I cannot do from the numbers.

What the numbers say:

	phones	duration	ms/phone	mean F0	wav RMS
T1	66	4.59 s	69.5	200.6 Hz	0.141
T2a	31	2.10 s	67.8	197.2 Hz	0.168
T2b	37	2.24 s	60.6	194.2 Hz	0.122
T3	31	1.94 s	62.5	201.7 Hz	0.154
T4	119	6.95 s	58.4	175.1 Hz	0.118
E1	85	5.84 s	68.7	244.6 Hz	0.088
E2a	50	4.08 s	81.5	239.2 Hz	0.122
E2b	56	3.70 s	66.1	243.4 Hz	0.088
E3	37	2.61 s	70.6	239.5 Hz	0.108
E4	78	5.62 s	72.0	245.7 Hz	0.088

Nothing is degenerate. Every sentence lands between 58 and 82 ms per phoneme, which is a normal speaking rate, no file clips, and both languages predict F0 inside their own corpus range: Telugu near its 187 Hz speaker mean, English near the average of its 115 speakers.

Pronunciation, sentence by sentence

E1, "antidisestablishmentarianism". The front end misses it in the lexicon and falls through to `g2p_en`, which produces `AE2 N T AY0 D IH0 S AH0 S T AE2 B L IH0 SH M AH0 N T EH1 R IY0 AH0 N IH0 Z AH0 M`. That is correct, and the 85-phone sequence holds together at 68.7 ms per phoneme, so the duration predictor does not fall apart on a sequence far longer than anything in training.

E2a and E2b, the "conduct" pair. Both come out as `K AA1 N D AH0 K T`, byte for byte identical, which is what section 1.6 predicted. The reason is worth restating precisely: ARPAbet carries stress in the phone symbol, so the noun is `AA1 ... AH0` and the verb should be `AA0 ... AH1`. The distinction lives in the phoneme string, not in the acoustics the model predicts on top of it. Since the front end hands the model the noun string for both, **no amount of training could fix E2b**. It is a front-end bug, not a model limitation.

E3, "The rhyth m". The deliberate break is preserved as instructed, and the consequence is visible: "rhyth" becomes `R AY1 DH` and "m" becomes `EH1 M`, so the model is synthesising something like "rythe em" rather than "rhythm". That is the correct behaviour given the assignment says to preserve the text.

E4, the date and time. `10:45` becomes `T EH1 N F AO1 R T IY0 F AY1 V`, `12` becomes `T W EH1 L V` and `2026` becomes `T W EH1 N IY0 T W EH1 N IY0 S IH1 K S`. All three are right, and all three were wrong before the fix in section 1.6.

T1, the long technical compound. 66 phones at 69.5 ms each. The Telugu G2P handles it without falling back to anything.

T2a and T2b, the పట్టు pair. Unlike English, both are genuinely the same word phonetically, so identical phones are correct here rather than a failure. Section 5 shows the prosody does differ.

T3, "పులి అడ విలో". The deliberate gap is preserved. The phones come out as `p u l i a dd a v i l oo`, and concatenating the two broken halves gives `a dd a v i l oo`, which is the correct pronunciation of the intended word అడవిలో. So the segmental content survives the break and only the word boundary, and whatever prosody the model attaches to it, is affected. That is a milder failure than E3, where the English break changed the phones themselves.

T4, the date and time in Telugu. 2026 becomes `r e nn dd u v ee l a i r a v ai aa r u`, 12వ becomes `p a n n e nn dd a v a` and 10:45 becomes `p a d i n a l a bh ai ai d u`. The Telugu normaliser handles all three, including the irregular ordinal.

---

5. Variance adaptor analysis

Three plots per sentence, in `project/Results/<language>/variance_adaptor_plots/{duration,pitch,energy}/`. Duration is in milliseconds, pitch denormalised back to Hz and energy to its original scale, all against the phoneme axis.

What the plots show

Pitch declines across the sentence. The clearest example is T2a, which starts near 195 Hz, peaks at 258 Hz on the first vowel, and falls to 155 Hz by the final phoneme. That is ordinary declination and the model has learned it.

Duration is quantised. Predicted durations land on multiples of 11.61 ms, because duration is an integer frame count at a 256-sample hop and 22.05 kHz. Values of 23.2, 34.8, 46.4, 58.0, 69.7 and 92.9 ms are 2, 3, 4, 5, 6 and 8 frames. This is a property of FastSpeech 2 rather than something learned, but it is visible in every duration plot and worth knowing when reading them.

Energy tracks vowels. In the T2 word పట్టు the vowels `a` and `u` carry energies of 82.6 and 58.7 while the stop closure `tt` sits at 5.99.

The T2 pair, where context does change the prediction

This is the interesting case, because the two sentences contain the identical word పట్టు with the identical phone string `p a tt tt u`, so any difference in the predictions comes from context alone.

phone	T2a duration	T2b duration	T2a F0	T2b F0	T2a energy	T2b energy
p	92.9 ms	104.5 ms	214.0 Hz	175.7 Hz	20.21	14.49
a	58.0	46.4	206.4	184.6	82.56	84.68
tt	69.7	69.7	192.7	192.6	19.18	17.48
tt	34.8	34.8	221.3	237.7	5.99	5.50
u	58.0	46.4	200.5	264.9	58.69	72.44
word	**313.5 ms**	**301.9 ms**	**207.0 Hz**	**211.1 Hz**		

The predictions differ, most sharply on the final `u`, 200.5 Hz against 264.9 Hz. So the variance adaptor is context sensitive, which it should be: self-attention runs over the whole phoneme sequence, so the same word gets different prosody depending on what surrounds it.

The honest reading is narrower than it first looks. పట్టు sits at phoneme 11 in T2a and phoneme 3 in T2b, so most of this difference is position in the sentence interacting with declination, not the model understanding that one means grip and the other means silk. It has no access to that distinction.

The E2 pair, where context changes very little that matters

phone	E2a duration	E2b duration	E2a F0	E2b F0
K	69.7 ms	104.5 ms	257.5 Hz	244.2 Hz
AA1	46.4	46.4	249.3	252.4
N	81.3	92.9	242.9	248.6
D	46.4	46.4	235.2	242.4
AH0	92.9	81.3	235.5	258.1
K	104.5	104.5	218.6	256.5
T	69.7	23.2	238.2	270.4
word	**510.8 ms**	**499.2 ms**	**239.6 Hz**	**253.2 Hz**

The same context sensitivity appears, but it cannot rescue the sentence. The stressed vowel `AA1` gets 46.4 ms in both and within 3 Hz of the same pitch, and `AA1` is on the **first** syllable in both. English marks the noun-verb contrast by moving primary stress to the second syllable, and that move is encoded in the phone symbols the front end chose, not in the duration and pitch the variance adaptor predicts on top of them. The adaptor is doing its job. It was handed the wrong input.

Reading these plots with section 3 in mind

Section 3 showed that the pitch and energy predictors fit their training data to within 0.02 and generalise 36 to 63 times worse. The contours above are plausible and internally consistent, and declination is clearly real, but they should be read as the output of predictors that have memorised a great deal of the training prosody. Duration is the component to trust most, especially for Telugu, where it is the only one that generalised.

6. Comparative observations between English and Telugu

Where they differ

Duration generalises for Telugu and not for English. Final validation duration loss is 0.075 for Telugu against 0.2297 for English, and the English figure never improves across the whole run. One speaker with one speaking rate is a learnable target. 115 speakers at different rates, with no speaker identity given at inference, is not.

Both overfit pitch and energy, and neither recovers. Telugu ends 61x and 42x worse on validation than training, English 36x and 63x. Both reach their best validation score within about 2000 steps.

Telugu needed far more work and produced the better-behaved model. Telugu had no G2P, no pronunciation dictionary and no acoustic model, so all three were built from scratch. English had all three, and still needed the dictionary extended for 4.53% of its tokens. Despite that, the Telugu model is the cleaner one, because the corpus is one speaker in consistent conditions and Svarah is an ASR benchmark recorded across 115 speakers and varying conditions. The dataset mattered more than the tooling.

Phonetically identical words behave differently in the two languages. T2's పట్టు really is the same word phonetically in both sentences, so identical phones are correct and the differing prosody in section 5 is a genuine, if positional, context effect. E2's "conduct" is **not** the same word phonetically, and producing identical phones is a failure.

Limitations

- **No part-of-speech input.** E2b is wrong and cannot be fixed by training. The fix is a front end that defers to `g2p_en` when the lexicon offers more than one pronunciation, which is a change I identified but did not make.
- **No speaker control for English.** A 115-speaker model with no speaker embedding at inference predicts an average voice and an average speaking rate.
- **80000 steps was the wrong target.** Both models converged inside 10000. Neither reached 80000, and section 3.3 shows the shortfall changed nothing.
- **The vocoder invents the 8 to 11 kHz band.** Both corpora are band-limited below 8 kHz by `mel_fmax`, and the universal HiFi-GAN reconstructs above that from a mel that never contained it.
- **Perceptual quality is unassessed.** See the note at the top of section 4.

What I would do differently

Early stopping on validation loss, which the curves make an obvious choice. A smaller model, since 35.2 M parameters against 5 hours of audio is the direct cause of the memorisation in section 3. A speaker embedding for English, conditioned at inference on a chosen speaker. And the `g2p_en` fallback for ambiguous lexicon entries, which is a few lines and would fix E2b outright.

7. Reproducibility

7.1 Dependencies

`requirements.txt` holds the pinned list. The ones that matter:

package	note
torch	2.x. CUDA, MPS and CPU all work via `device_utils.py`
librosa	0.10 or newer, for the keyword-only `pad_center`
pyworld	F0 extraction. Needs `setuptools<81`, since 84 removed `pkg_resources`
tgt	reads MFA TextGrids
g2p-en, inflect, unidecode	English front end
scikit-learn	the variance scalers
matplotlib, scipy, tqdm, pyyaml, tensorboard	plots, audio IO, logging

Montreal Forced Aligner is a separate conda install and is only needed to regenerate alignments. Version 3.4.3.

7.2 Commands

Data preparation:

```
python scripts/download_data.py
python scripts/prepare_telugu.py
python scripts/build_telugu_lexicon.py --rewrite-labs
```

Telugu alignment, which trains the acoustic model because MFA has no Telugu model:

```
mfa validate raw_data/Telugu lexicon/telugu_lexicon.txt
mfa train raw_data/Telugu lexicon/telugu_lexicon.txt \
    mfa_data/telugu_acoustic.zip mfa_data/textgrids_telugu --clean -j 8
```

English alignment, which downloads a pretrained model instead:

```
mfa model download acoustic english_us_arpa
mfa model download dictionary english_us_arpa
mfa align raw_data/English english_us_arpa english_us_arpa \
    mfa_data/textgrids_english --clean -j 8
```

Then per language, from `project/Telugu/` or `project/English/`:

```
python preprocess.py
python train.py --restore_step 0
python inference.py --restore_step 80000
```

Any config value can be overridden without editing the file, which is how the same config runs locally and on a hosted GPU:

```
python train.py --restore_step 0 \
    --set train.path.ckpt_path=/kaggle/working/ckpt \
    --set train.step.total_step=80000
```

Loss figures:

```
python scripts/plot_losses.py --language Telugu
```

7.3 Configuration files

One per language, `project/English/config.yaml` and `project/Telugu/config.yaml`. Each merges what the reference splits across three files, with a `preprocess`, a `model` and a `train` section. Paths inside are relative to the config file.

7.4 Checkpoints

Checkpoints are too large for plain git, so they are provided as described in `README.md`. Each save produces a full checkpoint with optimiser state, about 420 MB, and a weights-only copy, about 140 MB. The weights-only copy is what inference needs.

7.5 Notes on running this again

Two things that cost me time:

The inference and plotting path can be validated before any model is trained, by running `inference.py --restore_step 0` against a randomly initialised model. The audio is meaningless but it exercises G2P, symbol lookup, the model forward pass, the vocoder, wav writing and all three plots. I did this deliberately and it caught the vocoder path problem and the `weights_only` change before a long GPU run rather than after.

The reference repository's dependency pins cannot be satisfied on a current machine, and trying to install them into an active environment can upgrade packages that other work depends on. Create a dedicated environment and call its interpreter by absolute path.